Python OOP Advanced Guide

Magic Methods · Dunder Methods · Advanced Patterns

With Full Examples · When to Use · How to Use

Chapter 1 — OOP Quick Recap

Before diving into advanced OOP, let's confirm the foundation. In Python, everything is an object. A **class** is the blueprint; an **instance** is the real object created from it.

Basic Class Structure

```
class Animal:

# Class variable (shared by all instances)

kingdom = 'Animalia'


def __init__(self, name, sound): # Constructor

self.name = name # instance variable

self.sound = sound


def speak(self):

return f'{self.name} says {self.sound}'


dog = Animal('Rex', 'Woof')

print(dog.speak()) # Rex says Woof

print(dog.kingdom) # Animalia
```

Chapter 2 — Magic (Dunder) Methods

Dunder = Double UNDERscore. These are special methods Python calls *automatically* in response to operators, built-in functions, and language constructs. They let your classes behave like built-in types.

1. __init__ and __new__

__**new**__ creates the object; __**init**__ initialises it. You almost always only override __**init**__.

■ *When to use __new__: singletons, immutable subclasses (str, int), custom metaclass control.*

```
class Singleton:

_instance = None


def __new__(cls, *args, **kwargs):

if cls._instance is None:

cls._instance = super().__new__(cls)

return cls._instance


def __init__(self, value):

self.value = value


a = Singleton(10)

b = Singleton(99)

print(a is b) # True — same object!

print(b.value) # 99
```

2. __str__ and __repr__

__**repr**__: unambiguous developer representation (used in REPL/debugger). __**str**__: human-friendly string (used by print()).

■ *When to use: always define __repr__. Add __str__ when the human-readable version differs from the debug version.*

```
class Vector:

def __init__(self, x, y):

self.x, self.y = x, y


def __repr__(self):

return f'Vector({self.x}, {self.y})' # for devs
```

```
def __str__(self):

return f'({self.x}, {self.y})' # for users


v = Vector(3, 4)

print(v) # (3, 4) uses __str__

print(repr(v)) # Vector(3, 4) uses __repr__
```

3. Arithmetic Operator Methods

Override these to make your objects work with +, -, *, /, etc.

Method	Operator	Example
__add__	+	a + b
__sub__	–	a - b
__mul__	*	a * b
__truediv__	/	a / b
__floordiv__	//	a // b
__mod__	%	a % b
__pow__	**	a ** b
__neg__	-(unary)	-a
__abs__	abs()	abs(a)

```
class Vector:

def __init__(self, x, y): self.x, self.y = x, y


def __add__(self, other): # v1 + v2

return Vector(self.x + other.x, self.y + other.y)


def __sub__(self, other): # v1 - v2

return Vector(self.x - other.x, self.y - other.y)


def __mul__(self, scalar): # v * 3

return Vector(self.x * scalar, self.y * scalar)


def __abs__(self): # abs(v)

return (self.x**2 + self.y**2) ** 0.5
```

```
def __neg__(self): # -v

return Vector(-self.x, -self.y)


def __repr__(self): return f'Vector({self.x},{self.y})'


v1 = Vector(1, 2)

v2 = Vector(3, 4)

print(v1 + v2) # Vector(4,6)

print(v1 * 3) # Vector(3,6)

print(abs(v2)) # 5.0

print(-v1) # Vector(-1,-2)
```

4. Comparison Operator Methods

Make objects comparable with ==, !=, <, >, <=, >=.

■ *When to use: data classes, sorting, priority queues, custom ordering.*

```
from functools import total_ordering


@total_ordering # Auto-generates missing comparison methods

class Temperature:

def __init__(self, celsius):

self.celsius = celsius


def __eq__(self, other): # ==

return self.celsius == other.celsius


def __lt__(self, other): # <

return self.celsius < other.celsius


def __repr__(self):

return f'{self.celsius}°C'


temps = [Temperature(100), Temperature(0), Temperature(37)]

print(sorted(temps)) # [0°C, 37°C, 100°C]

print(Temperature(50) > Temperature(20)) # True
```

5. Container / Sequence Methods

Make your class behave like a list, dict or any container.

■ *When to use: custom data structures, wrappers around lists/dicts, domain-specific collections.*

```
class NumberBag:

def __init__(self):

self._data = []


def __len__(self): # len(bag)

return len(self._data)


def __getitem__(self, index): # bag[0]
```

```
return self._data[index]


def __setitem__(self, index, val): # bag[0] = 5

self._data[index] = val


def __delitem__(self, index): # del bag[0]

del self._data[index]


def __contains__(self, item): # 5 in bag

return item in self._data


def __iter__(self): # for x in bag

return iter(self._data)


def append(self, val):

self._data.append(val)


bag = NumberBag()

bag.append(10); bag.append(20); bag.append(30)

print(len(bag)) # 3

print(bag[1]) # 20

print(20 in bag) # True

for n in bag: print(n) # 10 20 30
```

6. Context Manager: __enter__ and __exit__

Enables the **with** statement. Guarantees setup and teardown even if an exception occurs.

■ *When to use: file handling, DB connections, locks, timers, any resource that must be released.*

```
import time


class Timer:

def __enter__(self): # Called on 'with' entry

self.start = time.time()

print('Timer started')

return self # becomes 'as' variable


def __exit__(self, exc_type, exc_val, exc_tb):

elapsed = time.time() - self.start

print(f'Elapsed: {elapsed:.4f}s')

return False # False = don't suppress exceptions


with Timer() as t:

total = sum(range(1_000_000))

# Output:

# Timer started

# Elapsed: 0.0312s
```

7. __call__ — Make Instances Callable

Lets you use an instance like a function: **obj()**.

■ *When to use: function decorators, ML models, factory patterns, memoization, configurable functions.*

```
class Multiplier:

def __init__(self, factor):

self.factor = factor


def __call__(self, value): # double(5)

return value * self.factor


double = Multiplier(2)
```

```
triple = Multiplier(3)

print(double(5)) # 10

print(triple(5)) # 15


# Works as a callback too!

nums = [1, 2, 3, 4]

print(list(map(double, nums))) # [2, 4, 6, 8]
```

8. Attribute Access: __getattr__, __setattr__, __getattribute__

__**getattr**__: called when attribute is NOT found normally. __**setattr**__: called on every attribute assignment. __**getattribute**__: called on every attribute access (use carefully!).

■ *When to use: proxy objects, validation on set, lazy loading, dot-access dict wrappers.*

```
class ValidatedModel:

_allowed = {'name': str, 'age': int, 'score': float}


def __setattr__(self, key, value):

if key in self._allowed:

expected = self._allowed[key]

if not isinstance(value, expected):

raise TypeError(f'{key} must be {expected.__name__}')

super().__setattr__(key, value) # MUST call super


def __getattr__(self, key): # Only if not found

return f'{key} not set'


m = ValidatedModel()

m.name = 'Alice' # OK

m.age = 30 # OK

# m.age = '30' # TypeError: age must be int

print(m.salary) # salary not set
```

9. __slots__ — Memory Optimisation

By default Python stores instance attributes in a __**dict**__. Using __**slots**__ replaces the dict with a fixed array, saving ~30–40% memory per instance.

■ *When to use: classes with millions of instances, data containers, performance-critical code.*

```
class Point: # WITHOUT __slots__

def __init__(self, x, y):

self.x, self.y = x, y


class FastPoint: # WITH __slots__

__slots__ = ['x', 'y']

def __init__(self, x, y):

self.x, self.y = x, y


import sys

p1 = Point(1, 2)

p2 = FastPoint(1, 2)

print(sys.getsizeof(p1.__dict__)) # ~232 bytes

print(sys.getsizeof(p2)) # ~56 bytes (no __dict__)


# FastPoint does NOT allow dynamic attributes:

# p2.z = 3 → AttributeError
```

10. Descriptors: __get__, __set__, __delete__

Descriptors are objects that control attribute access for other classes. Properties, classmethods, staticmethods are all descriptors internally.

■ *When to use: reusable validators, lazy computed attributes, ORM field definitions (like Django models).*

```
class PositiveNumber:

'''Reusable descriptor that enforces positive values'''

def __set_name__(self, owner, name): # Called at class creation

self.name = name


def __get__(self, obj, objtype=None): # obj.attr

if obj is None: return self
```

```
return obj.__dict__.get(self.name, 0)


def __set__(self, obj, value): # obj.attr = val

if value <= 0:

raise ValueError(f'{self.name} must be positive')

obj.__dict__[self.name] = value


class Circle:

radius = PositiveNumber() # Descriptor instance

area = PositiveNumber()


def __init__(self, radius):

self.radius = radius # triggers __set__


c = Circle(5)

print(c.radius) # 5

# c.radius = -1 # ValueError: radius must be positive
```

11. Iterator Protocol: __iter__ and __next__

Make your class iterable (usable in for loops, list(), zip(), etc.).

■ *When to use: custom data streams, infinite sequences, lazy evaluation, file-like objects.*

```
class Fibonacci:

'''Infinite Fibonacci iterator'''

def __init__(self):

self.a, self.b = 0, 1


def __iter__(self): # Returns the iterator object (self)

return self


def __next__(self): # Returns next value

value = self.a

self.a, self.b = self.b, self.a + self.b

return value


fib = Fibonacci()

first10 = [next(fib) for _ in range(10)]

print(first10) # [0,1,1,2,3,5,8,13,21,34]


# Or use itertools.islice to limit infinite iterator:

from itertools import islice

print(list(islice(Fibonacci(), 8))) # [0,1,1,2,3,5,8,13]
```

12. __hash__ — Use Objects in Sets / Dict Keys

Objects need **__hash__** to be stored in sets or used as dict keys. If you define **__eq__**, Python sets **__hash__ = None** — you must define it manually.

■ *When to use: whenever you need objects as dict keys or in sets (cache keys, deduplication, lookup tables).*

```
class Point:

def __init__(self, x, y):

self.x, self.y = x, y


def __eq__(self, other):
```

```
return self.x == other.x and self.y == other.y


def __hash__(self): # REQUIRED if __eq__ defined

return hash((self.x, self.y))


def __repr__(self):

return f'Point({self.x},{self.y})'


points = {Point(1,2), Point(3,4), Point(1,2)} # set

print(points) # {Point(1,2), Point(3,4)} dedup!


cache = {Point(0,0): 'origin', Point(1,0): 'unit-x'}

print(cache[Point(0,0)]) # 'origin'
```

Chapter 3 — Inheritance, MRO & Mixins

Python supports multiple inheritance. The **Method Resolution Order (MRO)** determines which method is called. Use **super()** to delegate up the chain.

Multiple Inheritance & MRO

```
class A:

def greet(self): return 'Hello from A'


class B(A):

def greet(self): return 'Hello from B'


class C(A):

def greet(self): return 'Hello from C'


class D(B, C): # Diamond inheritance

pass


d = D()

print(d.greet()) # Hello from B (MRO: D→B→C→A)

print(D.__mro__) # (, , , )
```

Mixins — Reusable Behaviour Without Full Inheritance

Mixins are small classes designed to add specific behaviour. They are never instantiated alone.

■ *When to use: logging, serialisation, validation, repr generation — any cross-cutting concern.*

```
class JSONMixin:

import json

def to_json(self):

import json

return json.dumps(self.__dict__, indent=2)


class ReprMixin:

def __repr__(self):

attrs = ', '.join(f'{k}={v!r}' for k,v in self.__dict__.items())

return f'{type(self).__name__}({attrs})'
```

```
class User(JSONMixin, ReprMixin):

def __init__(self, name, email):

self.name = name

self.email = email


u = User('Alice', 'alice@example.com')

print(u) # User(name='Alice', email='alice@example.com')

print(u.to_json()) # {"name": "Alice", "email": "alice@..."}
```

Chapter 4 — Properties, Class Methods, Static Methods

@property — Computed Attributes with Validation

@property turns a method into a read-only attribute. Add a setter for writable properties with validation.

■ *When to use: validation on assignment, computed attributes, backward-compatible refactoring.*

```
class Circle:

def __init__(self, radius):

self._radius = radius # private


@property

def radius(self): # getter

return self._radius


@radius.setter

def radius(self, value): # setter with validation

if value < 0:

raise ValueError('Radius cannot be negative')

self._radius = value


@property

def area(self): # computed, read-only

import math

return math.pi * self._radius ** 2


c = Circle(5)

print(c.area) # 78.53...

c.radius = 10

print(c.area) # 314.15...

# c.radius = -1 # ValueError
```

@classmethod and @staticmethod

```
class Date:
```

```
def __init__(self, year, month, day):

self.year, self.month, self.day = year, month, day


@classmethod

def from_string(cls, date_str): # Alternative constructor

y, m, d = map(int, date_str.split('-'))

return cls(y, m, d) # uses cls, not Date


@staticmethod

def is_valid(date_str): # No self or cls

try:

y, m, d = map(int, date_str.split('-'))

return 1 <= m <= 12 and 1 <= d <= 31

except:

return False


def __repr__(self):

return f'{self.year}-{self.month:02d}-{self.day:02d}'


d = Date.from_string('2025-07-20') # classmethod

print(d) # 2025-07-20

print(Date.is_valid('2025-13-01')) # False (staticmethod)
```

Chapter 5 — Abstract Base Classes (ABC)

ABCs define an interface that subclasses MUST implement. Trying to instantiate a class with unimplemented abstract methods raises a TypeError.

■ *When to use: plugin systems, strategy patterns, enforcing contracts in team code, any place where you want to guarantee subclasses implement specific methods.*

```
from abc import ABC, abstractmethod


class Shape(ABC): # Abstract base


@abstractmethod

def area(self) -> float: # Must be overridden

...


@abstractmethod

def perimeter(self) -> float:

...


def describe(self): # Concrete method (shared)

return f'{type(self).__name__}: area={self.area():.2f}'


class Rectangle(Shape):

def __init__(self, w, h): self.w, self.h = w, h

def area(self): return self.w * self.h

def perimeter(self): return 2*(self.w + self.h)


class Circle(Shape):

def __init__(self, r): self.r = r

def area(self): return 3.14159 * self.r**2

def perimeter(self): return 2 * 3.14159 * self.r


shapes = [Rectangle(4,5), Circle(3)]

for s in shapes:

print(s.describe())
```

```
# Rectangle: area=20.00

# Circle: area=28.27


# Shape() ← TypeError: Can't instantiate abstract class
```

Chapter 6 — Dataclasses (Python 3.7+)

@dataclass auto-generates __init__, __repr__, __eq__ and more from field annotations. Less boilerplate, same power.

■ *When to use: data containers, config objects, DTOs, anywhere you would otherwise write repetitive __init__ code.*

```
from dataclasses import dataclass, field, KW_ONLY

from typing import List


@dataclass(order=True) # generates comparison methods

class Student:

sort_index: float = field(init=False, repr=False)

name: str

grade: float

courses: List[str] = field(default_factory=list)


def __post_init__(self): # Called after __init__

self.sort_index = self.grade


s1 = Student('Alice', 9.5, ['Math', 'CS'])

s2 = Student('Bob', 8.0)

s3 = Student('Cara', 9.8)


print(s1) # Student(name='Alice', grade=9.5, ...)

print(s1 == s1) # True

print(sorted([s1,s2,s3])) # sorted by grade
```

Quick Reference — All Magic Methods

Method(s)	Triggered by	Use case
__init__	MyClass()	Initialise instance
__new__	MyClass()	Control object creation (Singleton)
__repr__ / __str__	repr() / str()	Dev / user string output
__add__, __sub__, __mul__	+ - *	Arithmetic on objects
__eq__, __lt__, __le__	== < <=	Comparison, sorting
__len__	len(obj)	Size of container
__getitem__, __setitem__	obj[key]	Index / slice access
__contains__	x in obj	Membership test
__iter__, __next__	for x in obj	Custom iteration
__enter__, __exit__	with obj:	Resource management
__call__	obj()	Callable instances
__getattr__, __setattr__	obj.attr	Attribute control
__hash__	hash(obj)	Dict keys / sets
__slots__	class definition	Memory optimisation
__get__, __set__	descriptor access	Reusable validators / ORM

■ *You now know Advanced Python OOP! Practice daily and you'll master it.*